

Knowledge Tree

Knowledge Tree is abbreviated as KT in this document.

Hardware



Power source for USB hub

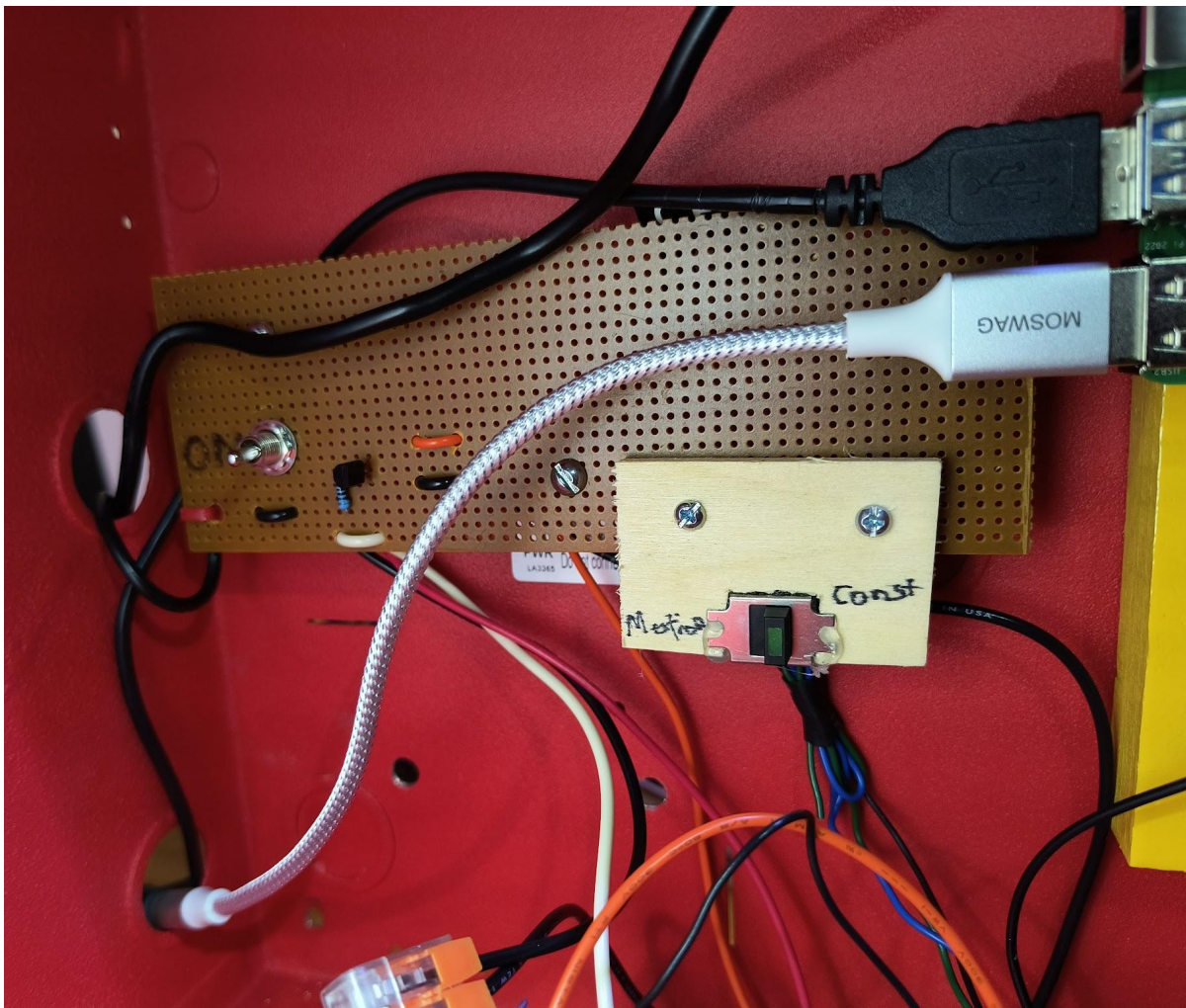
Power source for Raspberry Pi 5

Note: The RP5 requires alot of power. Its recommended to use the official power source.

This is the audio output. It should be plugged into an amplified speaker.

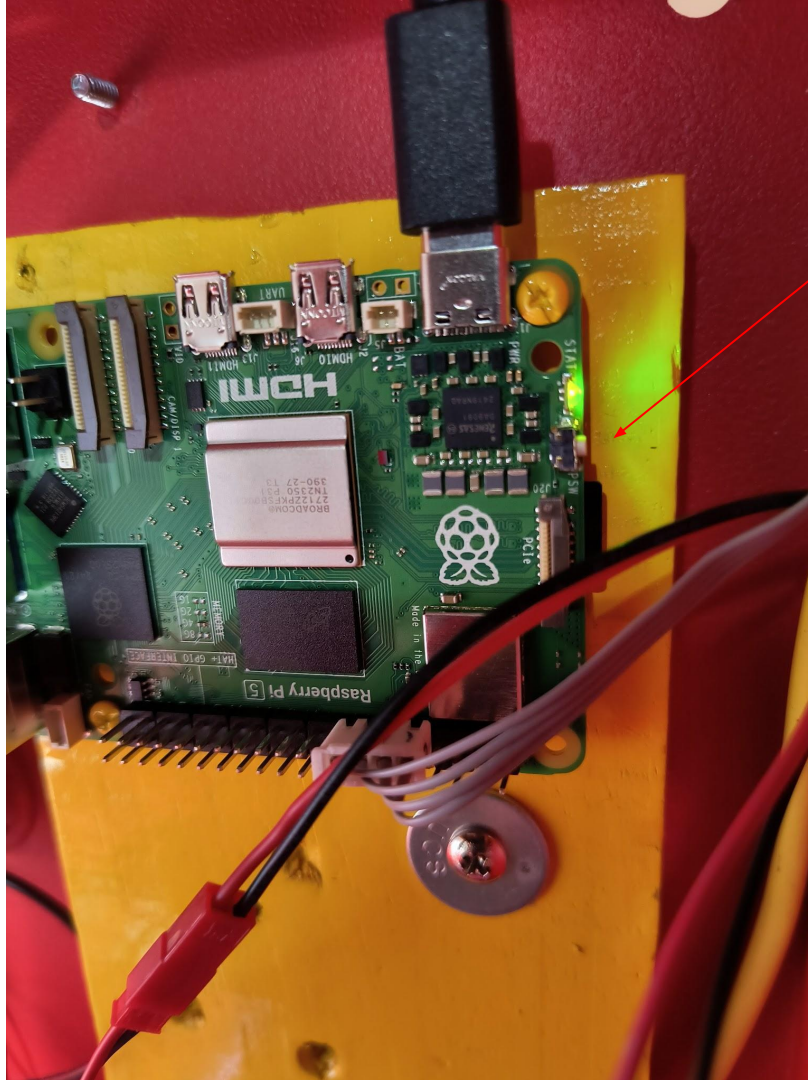
The RPi5 does not have a 3.5 mm audio port. This audio cable is connected to a USB to 3.5 mm converter.





This switch can be left on “Const”. That will leave the backlight in the KT on constantly.

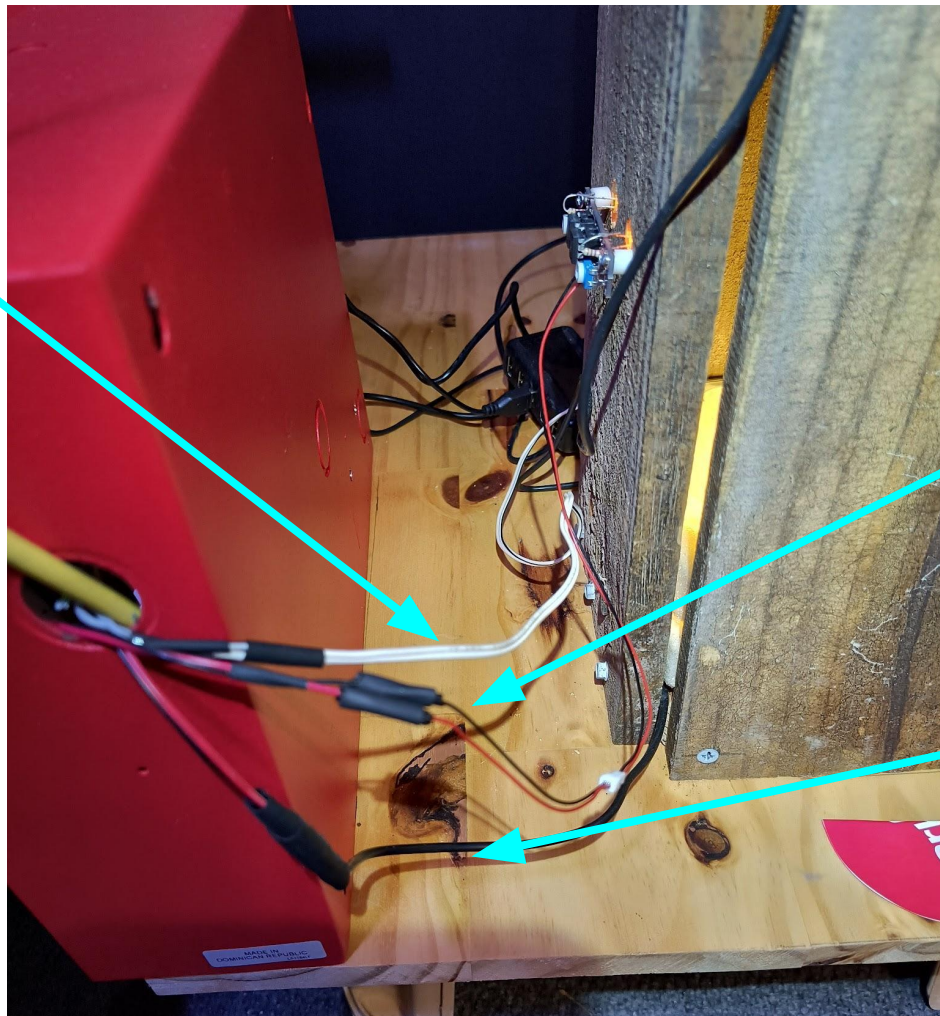
I originally wanted to have the backlight be motion-activated. It works, but the light often turns off too quickly. Moving the switch to the left will put it in motion-activated mode.



The RPi5 will start up automatically when its power supply is first connected. However, there is a on/off button here.

This connector goes to the pushbutton that starts the candy inspection. It is wired to the GPIO 17 (I think).

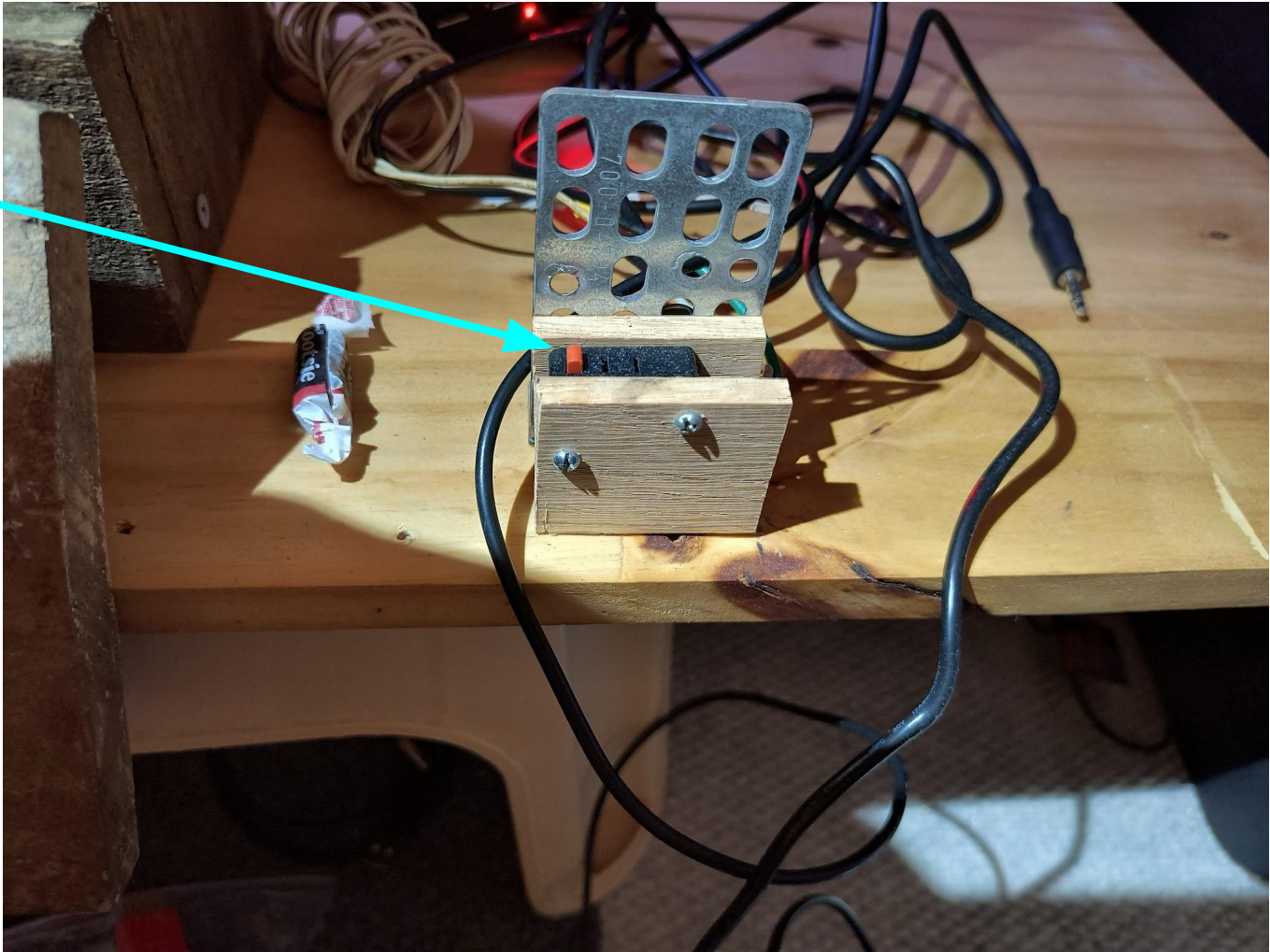
Wire to
pushbutton.



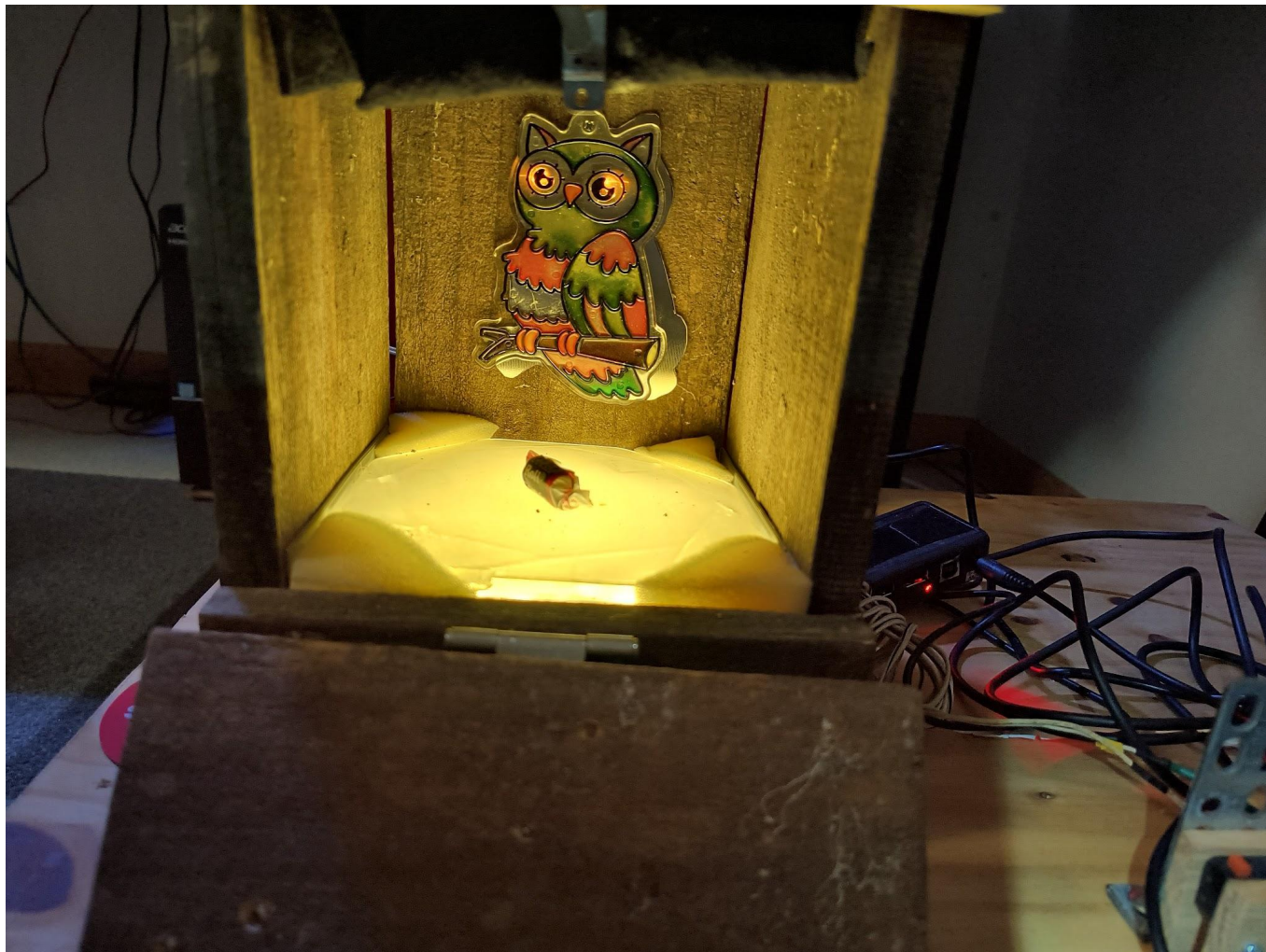
Wire to owl
eyes.

Wire to
backlight.

Pushbutton



Backlight with
candy on it.



Software

Connecting to the RPi5:

You can use any SSH client or VNC client. There are other tools such as Pi Finder that can be used.

The ethernet port is set to use the static IP address: 192.168.1.7

You can also connect over Wifi but it requires configuration. So you would need to first connect over ethernet cable and configure the Wifi connection. Or there is a way to configure the Wifi settings by plugging the SD card into a computer and modifying a settings file.

Here is an example of connecting over ethernet using TigerVNC viewer:
I connected an ethernet cable between my computer and the RPi5.
I set my computer's ethernet port to 192.168.1.99 and Subnet mask 255.255.255.0.
You don't have to use that exact IP address but it needs to be in the same network as the RPi5.

Edit IP settings

Manual

IPv4

☒ On

IP address

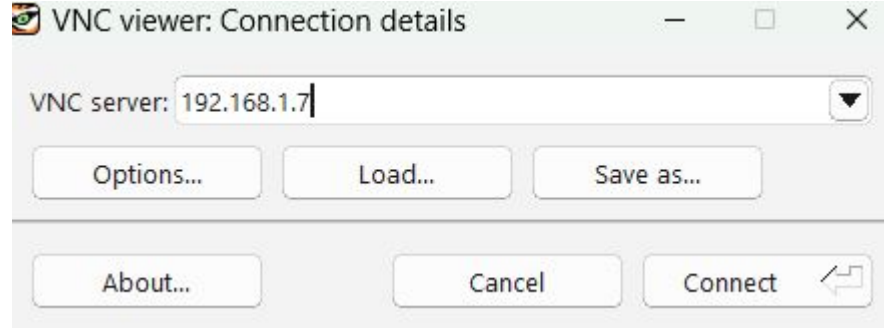
192.168.1.99

Subnet mask

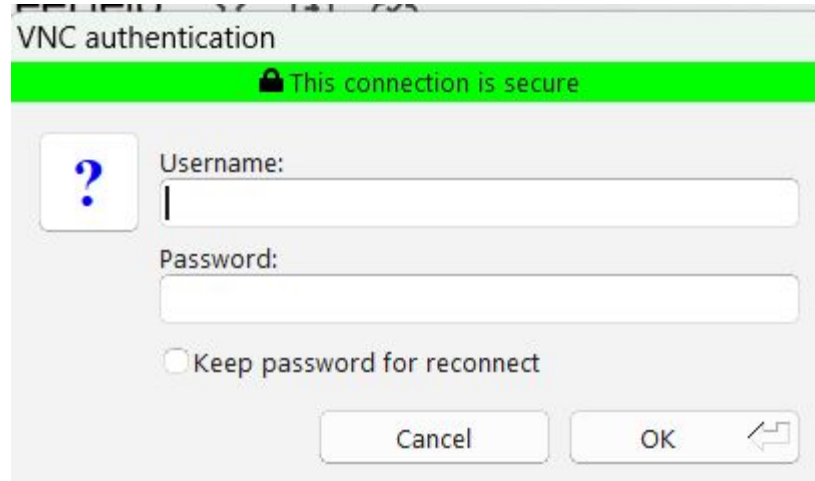
255.255.255.0

Gateway

Here is an example of connecting over ethernet using TigerVNC viewer:
Open TigerVNC viewer and type in the IP address of the RPi5.
Click Connect.



When prompted for authentication use rpi and rpi



A screenshot of a VNC authentication dialog box. The title bar reads "VNC authentication". Below the title bar is a green status bar with a lock icon and the text "This connection is secure". The main area is light gray and contains a blue question mark icon in a white box on the left. To the right of the icon are two text input fields: "Username:" and "Password:". Below the password field is a checkbox labeled "Keep password for reconnect". At the bottom are two buttons: "Cancel" and "OK" with a return key icon.

VNC authentication

🔒 This connection is secure

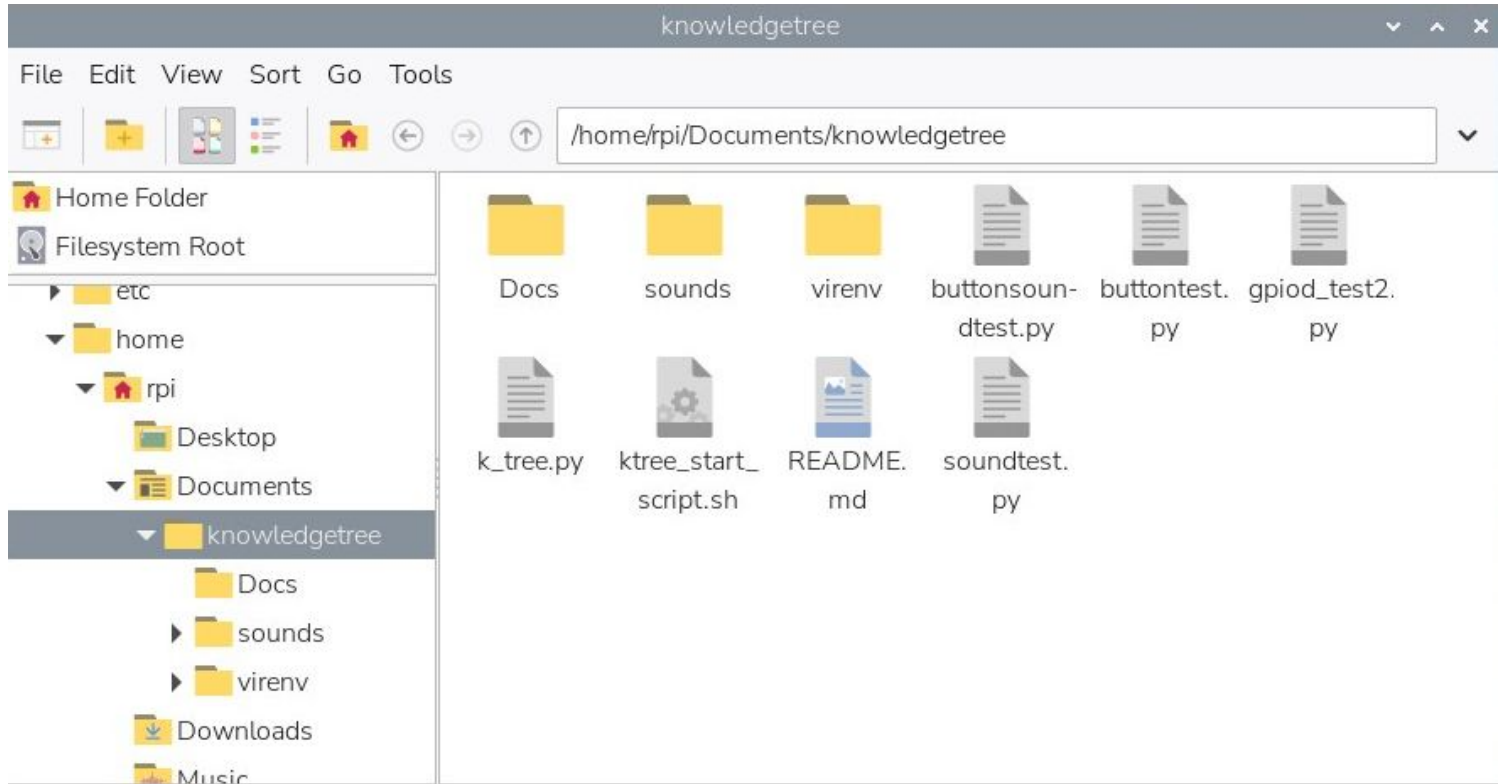
? Username:

Password:

☐ Keep password for reconnect

Cancel OK ↵

The repo is located in /home/rpi/Documents/knowledgetree
k_tree.py is main code file. The other files were used for testing.



The program starts automatically when the RPi5 boots up via a systemd service.
The systemd service file is saved in:
`/etc/systemd/system/knowledge-tree.service`

Note: If you want to test the program you may need to stop the systemd service so that its not running a duplicate instance of the program.
Refer to `K_Tree_README_Programming.txt` for instructions on starting/stopping a systemd service.

This is the contents of `knowledge-tree.service`:

```
[Unit]
Description=knowledgetree

[Service]
User=rpi
ExecStart=/home/rpi/Documents/knowledgetree/virenv/bin/python3 /home/rpi/Documents/knowledgetree/k_tree.py

[Install]
WantedBy=multi-user.target
```

The program is run by python in a virtual environment.

The virtual environment is located here:

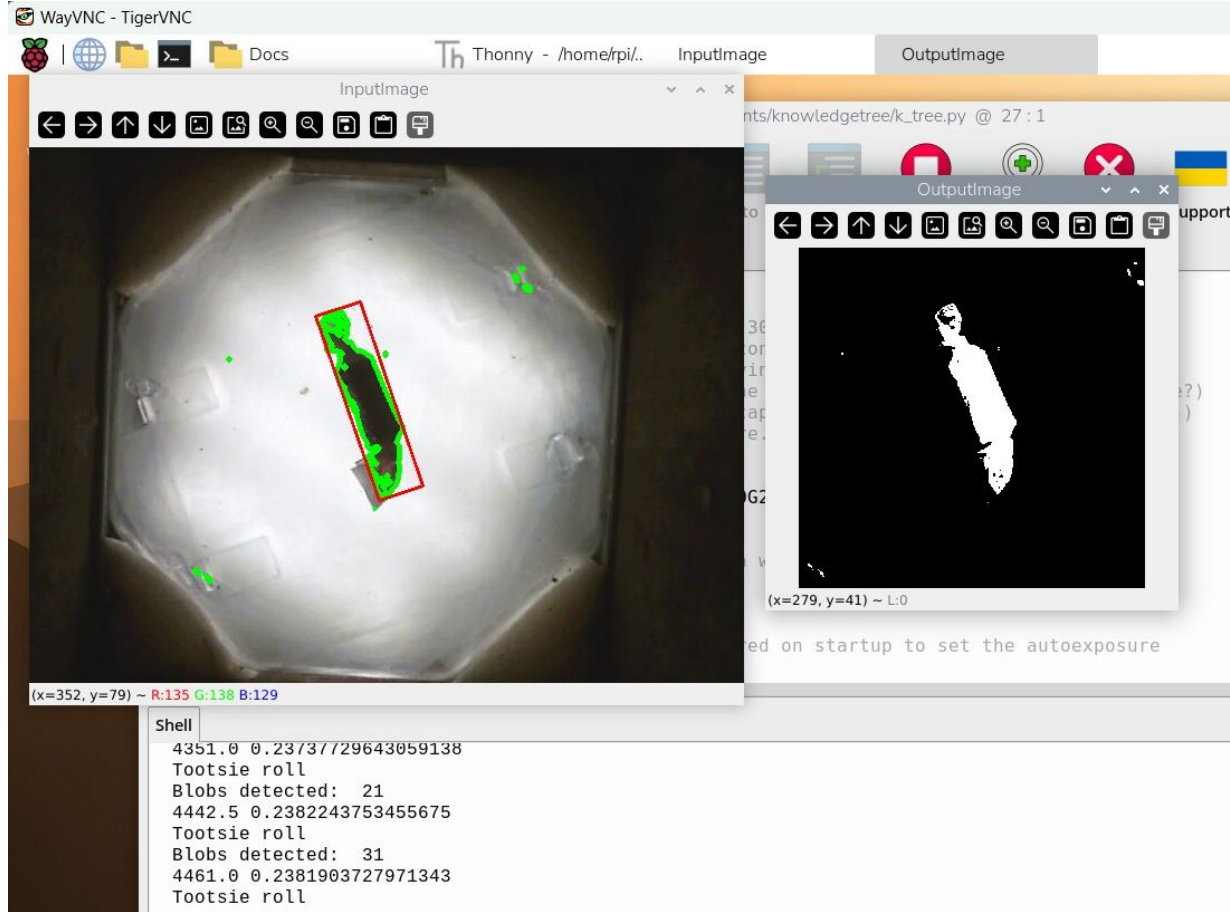
`/home/rpi/Documents/knowledgetree/virenv/bin`

The program can be put in test mode. This will display the camera images and the blob values that are detected.

k_tree.py ✕

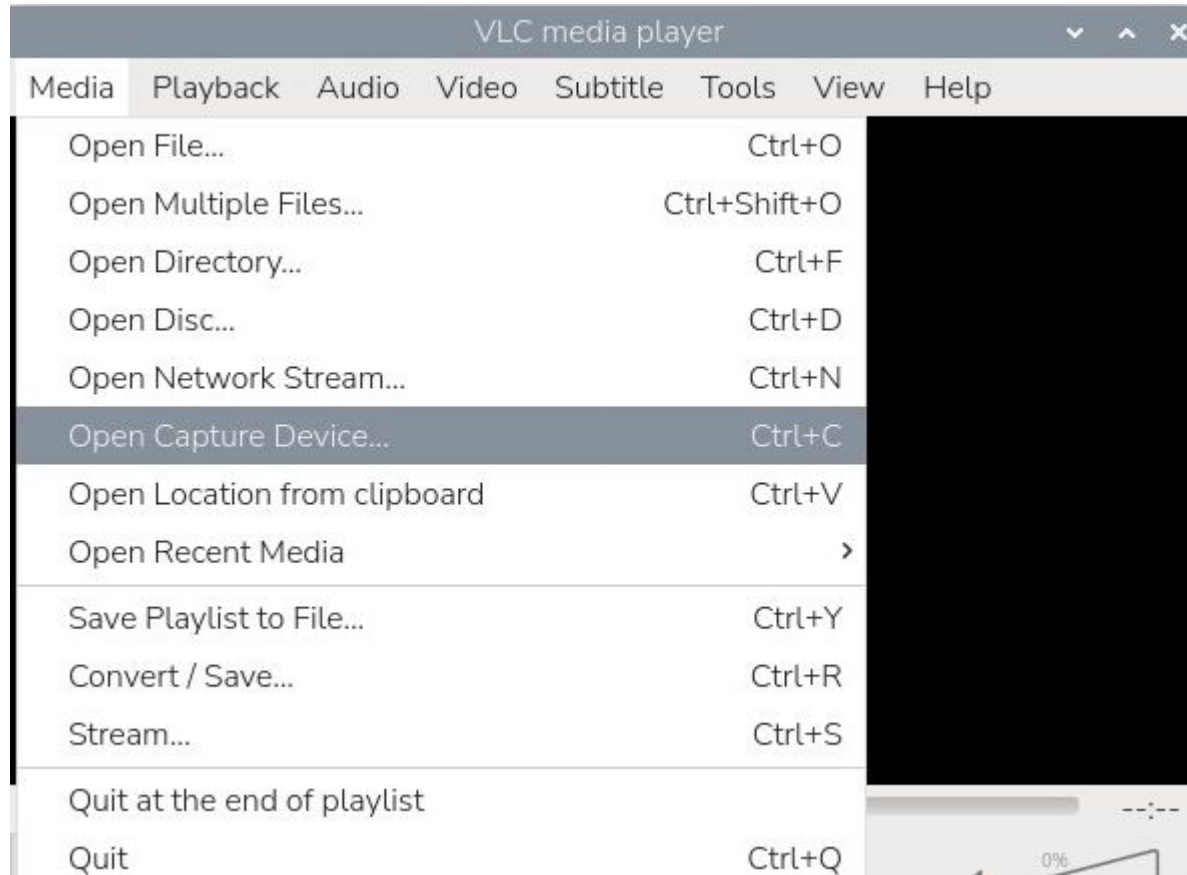
```
17 #This camera does not seem to support Exposure control. It always does auto-exposure
18 #capture.set(cv2.CAP_PROP_AUTO_EXPOSURE, 1) #Trying to turn off auto-exposure
19 #capture.set(cv2.CAP_PROP_EXPOSURE, -10) #Set the exposure (Whats the unit for exposure?)
20 #print("CAP_PROP_AUTO_EXPOSURE : '{}'.format(capture.get(cv2.CAP_PROP_AUTO_EXPOSURE)))
21 #print("CAP_PROP_EXPOSURE : '{}'.format(capture.get(cv2.CAP_PROP_EXPOSURE)))
22
23 # Background subtractor
24 bg_subtractor = cv2.createBackgroundSubtractorMOG2()
25
26 # Current mode
27 #mode = "test" #In this mode, the candy ID data will be displayed on screen
28 mode = "normal"
```

This image shows the program running in “test” mode.

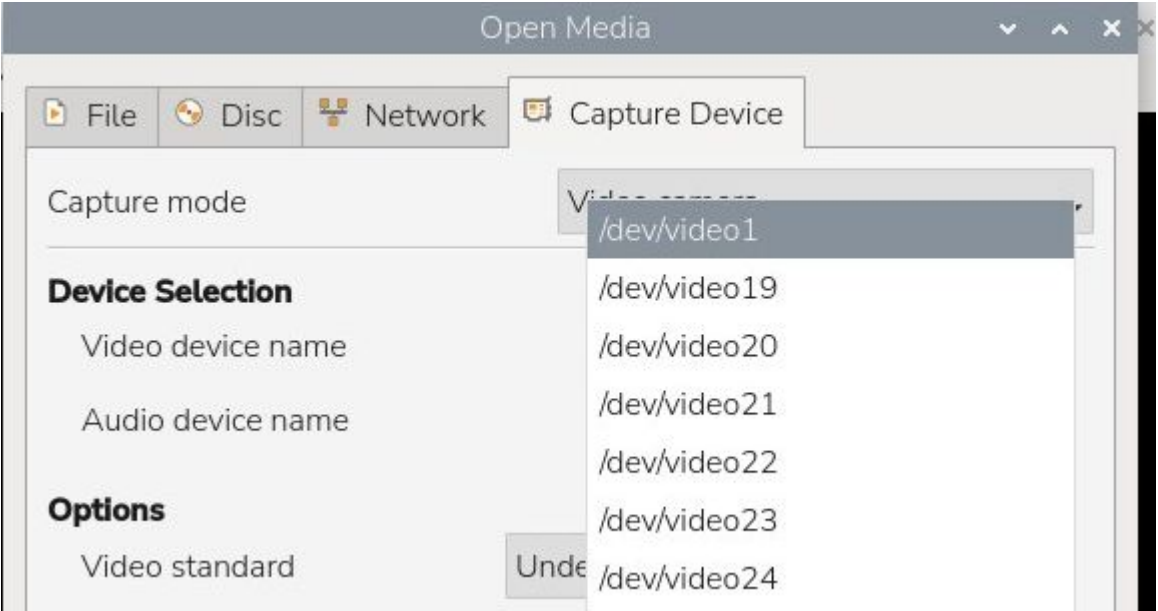


Testing the camera

VLC media player can be used to test the camera.
Open VLC and then click on Media-> Open Capture Device

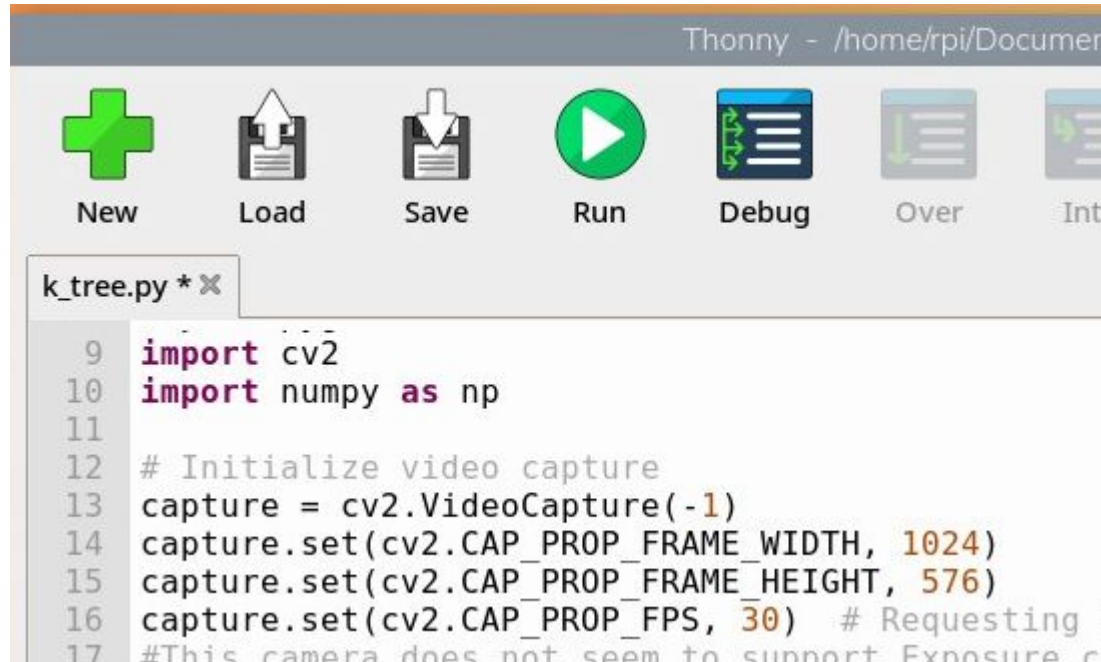


Look at the list of video sources. The camera is usually video0 or video1.
Click on video0 or video1 and then click Open.
If you see the video open correctly, then that is the correct video source.



In the `k_tree.py` program, the video source is set on line 13.

In this example, I used `-1` which means use the default source. If necessary, you can replace the `-1` with `0` or `1` depending on the video source.



The screenshot shows the Thonny IDE interface. The title bar reads "Thonny - /home/rpi/Document". The toolbar contains icons for New (green plus), Load (floppy disk with up arrow), Save (floppy disk with down arrow), Run (green play button), Debug (blue bug), Over (grey down arrow), and Int (grey left arrow). Below the toolbar, a tab labeled "k_tree.py * x" is active. The code editor displays the following Python code:

```
9 import cv2
10 import numpy as np
11
12 # Initialize video capture
13 capture = cv2.VideoCapture(-1)
14 capture.set(cv2.CAP_PROP_FRAME_WIDTH, 1024)
15 capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 576)
16 capture.set(cv2.CAP_PROP_FPS, 30) # Requesting
17 #This camera does not seem to support Exposure c
```


Additional information is available here:
</home/rpi/Documents/knowledgetree/Docs>